

RMC — MC-Core Block Reference

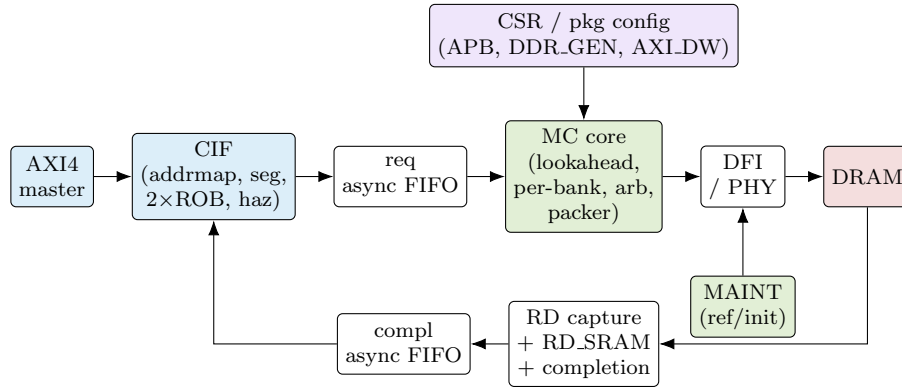
sub-block structure, I/O ports, and functions

Reconfigurable Memory Controller (DDR1–5), one channel per core

October 6, 2026

This reference reflects the current RTL (`rtl/cif`, `rtl/utlis`, `rtl/mc`, `rtl/rtl_pkg_config`, `rtl/csr_config`). Built blocks list real ports; not-yet-RTL blocks are described at block level. One MC core per DDR channel; channel select is extracted in the CIF.

1 Top spine



Forward: AXI → CIF → req async FIFO → lookahead → bank-dec → per-bank cell → arb → packer → DFI → DRAM. Return: DRAM → RD capture → RD_SRAM → completion → compl async FIFO → CIF. Only true CDC = CIF↔MC (the two async FIFOs); DFI seam is same-clock.

2 Compile-time config (`rmc_cfg_pkg`)

One DDR generation per build (`DDR_GEN`). Everything parametric.

param	value (DDR5)	meaning
DDR_GEN	DDR5	build-time generation select (DDR1–5)
N_RANKS	2	ranks (lanes); fully duplicated state, shared issue
N_BG /	8 / 4	bank groups / banks per group (= 32 banks/rank)
N_BANK_PER_BG		
ROW_W / COL_W	18 / 10	row / column address widths
BL / DQ_W	16 / 32	burst length / sub-channel data width
PKT_BYTES	64	packet = BL*DQ_W/8 = one 64 B line = one CAS
N_CH	2	DDR channels (one MC core each)
DDR_CHANNEL_W	2*DQ_W=64	data width per DDR channel
AXI_DW	N_CH*DDR_CHANNEL_W	matched BW (see §8)
P_MAX	4	packets per bank (row-hit train); sets bank queue depth
BANK_Q_DEPTH	P_MAX=4	per-bank TCAM / cas_fifo depth
DADDR_W	34	{rank,bg,bank,row,col} DRAM-coord width

3 Address map (STAGE 24)

Pure field-slice, no hash (`rmc_cif_addrmap`). Consecutive packets rotate BGs (diff-BG CAS = $t_{CCD,S} = 8$, writes safe), then `col_lo` revisits a bank's next column = row-hit.

Interleave (LSB→MSB): `offset[5:0]` | `ch` | `rank` | `BG_lo(rot)` | `col_lo(=P)` | `BG_hi` | `bank` | `col_hi` | `row`.

request	packets	/ch	/(ch,rank)	banks	P/bank	cols (row-hit)
4 KB (MAX)	64	32	16	4	4	4
2 KB	32	16	8	4	2	2
1 KB	16	8	4	4	1	1
64 B	1	1	1	1	1	1

`rmc_cif_addrmap` ports: `sys_addr` → `daddr{rank,bg,bank,row,col}`, `ch` (route to per-channel core), `offset` (byte-in-packet, write mask).

4 CIF — client interface

Between AXI4 and the MC async FIFOs. Owns address map, segmentation, two ROBs, and the same-line hazard interlock. Completion never back-pressures (`ready` tied high).

4.1 `rmc_cif` (top)

port	dir	function
<code>aclk</code> , <code>aresetn</code>	in	AXI clock / async reset
<code>aw*/w*/b*</code> AXI4	io	write address / data / response channels
<code>ar*/r*</code> AXI4	io	read address / data channels
<code>async_mc_req_*</code>	out	req stream to MC (<code>rob_index</code> , <code>op</code> , <code>daddr</code> , <code>pkt_num</code> , <code>last_in_txn</code> , <code>sram_slot</code>)
<code>async_mc_cmpl_*</code>	in	completion from MC (<code>rob_index</code> , <code>pkt_num</code> , <code>status</code>); <code>ready</code> tied high
<code>wdb_we/waddr/wdin</code>	out	WD_SRAM write port (CIF writes W beats)
<code>rdb_re/raddr/rdout</code>	out/in	RD_SRAM read port (CIF drains to R)

4.2 Sub-blocks

- **`rmc_cif_addrmap`** — byte `addr` → `daddr` + `ch` + `offset` (§3).
- **`rmc_cif_seg`** — burst → ≤16-beat packets, narrow/WSTRB flag (stub).
- **`rmc_cif_rob`** (×2, `r_rob/w_rob`) — order via pointer shift-reg, index-mapped backing stores (`addr` TCAM, inflow/outflow counters, cold meta). `Addr-TCAM` search port probed by the opposite ROB.
- **`rmc_cif_haz`** — RAW+WAR interlock. 1-cycle hits latch a stall on the requester's slot; released when the matched older entry retires. WAW impossible (AXI write ordering).

`rmc_cif_haz` ports:

port	dir	function
<code>clk</code> , <code>rst_n</code>	in	stateful (latched stalls)
<code>raw_hit</code> , <code>raw_rd_ptr</code>	in	new read overlaps older write; new read's slot to hold
<code>raw_wr_ptr</code>	in	matched older write slot (release key)
<code>war_hit</code> , <code>war_wr_ptr</code> , <code>war_rd_ptr</code>	in	symmetric WAR
<code>wr_free_vld/ptr</code> , <code>rd_free_vld/ptr</code>	in	retire pulses -i release matching stalls

port	dir	function
r_stall_vector, w_stall_vector	out	held per-entry stall bits to the two ROBs

5 Utility library (rtl/utlis)

Generic, parametric, standalone.

module	function + ports
rmc_fifo	sync FIFO (count-based). Params WIDTH,DEPTH,FWFT,AF/AE_THRESH,PROT. Ports: clk,rst_n, wr_en,wr_data, rd_en,rd_data,rd_valid, full,empty,almost_*,count,overflow,underflow.
rmc_sram	1W+1R RAM. Params WIDTH,DEPTH,RD_LAT(0/1/2),COLLIDE. RD_LAT=0=comb (0th-clk), write-first. Ports: clk, we,waddr,wdata, re,raddr,rdata.
rmc_regfile	flop array, 1 write + N comb read ports, write-first bypass. Ports: clk,rst_n, we,waddr,wdata, raddr[N_RD],rdata[N_RD].
rmc_async_fifo	dual-clock Gray-pointer CDC FIFO. Ports: wclk,wrst_n,wr_en,wr_data,wfull,walmost_full, rclk,rrst_n,rd_en,rd_data,rd_valid,rempty,ralmost_empty.
rmc_lookahead	bank-stall bypass window (below).

5.1 rmc_lookahead — bank-stall bypass window

DEPTH-entry age-ordered window. Oldest-*eligible* (valid & rdy_vec[adr]) issues, so a younger packet to a ready resource bypasses a stalled older one; same-resource order held by the shared ready bit. Payload in reg (PAYLOAD-IN_REG=1) or in a comb-read meta store (rmc_sram RD_LAT=0, write-first) + free-list. Same-cycle arrive+issue: din issues live (bypass), not stored; the wr_ptr==rd_ptr compare (internal to the write-first store) picks live vs stored.

port	dir	function
clk, rst_n	in	
din_valid/rdy_out	in/out	ingress handshake (rdy = window has room)
din_adr	in	resource key (probes rdy_vec); = {bg,bank}
din_payload	in	wide payload (= {r/w,row,col,mem_ptr})
rdy_vec[N_RDY]	in	per-resource ready (= bank_rdy)
out_valid/ready	out/in	egress handshake
out_adr,	out	issued packet
out_payload		

6 Per-bank path (rmc_bank)

One cell per {rank,bank}. Open-page, single open row. Exploits the row-streaked arrival from the address map: two comparators + a 1-bit toggle split the stream.

- **CMP1** row == open_row (& state==OPEN) → hit → cas_fifo; else miss.
- **CMP2** row == last_miss_row → new_row flips the toggle; miss pushed to row_miss_fifo tagged with the toggle.
- **toggle** delimits per-row groups in row_miss_fifo: drain pops while toggle==head.toggle = one row → one ACT, then CAS the group.
- depths: cas_fifo = row_miss_fifo = BANK_Q_DEPTH = P_MAX = 4.
- FSM: IDLE → ACTING $\xrightarrow{t_{RCD}}$ OPEN $\xrightarrow{grant_pre}$ PREING $\xrightarrow{t_{RP}}$ IDLE, plus REFING. Issue edges start transitions; scoreboard can_* complete the transient states.

port	dir	function
clk, rst_n	in	
admit_valid/ready	in/out	packet in from decoder (post-lookahead)
admit_row/col/op/sptr	in	row, column, R/W, payload slot
can_act/cas/pre	in	timing legality from scoreboard
ref_pending/ref_done	in	maintenance refresh handshake
act_rdy/cas_rdy/pre_rdy	out	command readiness to the arb tree
grant_act/cas/pre/ref	in	arb grant (pop on CAS)
out_row/col/op/sptr	out	issued-command payload to packer

7 Back-end (planned RTL)

Block-level; not yet implemented.

- **SCOREBOARD** — GC (CK counter), per-scope timing regs; `next_* = GC + const`; comparator $\rightarrow$ `can_*` ((GC-next) [MSB]). Split by arb level (bank/BG/rank/global); one shared `TIMING_CONST` addend table. `dqFree` gates CAS (cross-rank occupancy).
- **ARB** class-split lanes (cas/act/pre) per rank $\rightarrow$ L3 2:1-per-class across ranks $\rightarrow$ N-winner phase bundle. Selection+fairness only (legality folded into `can_*`).
- **PHASE PACKER** — gear 1:N, places the bundle (CA slot, ≤ 1 CAS/8CK), sole FSM-commit edge; arms the data engines at CAS-commit.
- **DATA** — WL launcher + write splitter ($\leftarrow$ WD_SRAM), RL line + read accumulator ($\rightarrow$ RD_SRAM); deterministic shift lines, no CAM.
- **COMPLETION** — merges rd_done/wr_done $\rightarrow$ tag-only {rob_index, pkt_num, status} $\rightarrow$ resp async FIFO.
- **MAINT** — INIT + REFRESH (REFsb-preferred, debt counter), RFM/ZQ/MRW stubs; override-inject into the packer, never CAS.

8 Matched-bandwidth AXI width

$$\text{AXI_DW} = \text{N_DDR_CHANNELS} \times \text{DDR_CHANNEL_WIDTH}$$

so AXI ingress BW = DDR egress BW $\Rightarrow$ WD/RD SRAM and ROB inflow/outflow stay balanced (no buffer blow-up). Derived in the package, not a free per-instance parameter.

Caveat: bandwidth = width $\times$ frequency, so the width-only form is exact *only* when `aclk == DDR data rate`. If the clocks differ, scale:

$$\text{AXI_DW} = \text{N_CH} \times \text{DDR_CHANNEL_W} \times \frac{f_{ddr}}{f_{aclk}},$$

else AXI bottlenecks and the buffers drift (`CLK_RATIO` param = TODO). DDR sustained BW is $\sim 60\text{--}85\%$ of peak, so AXI sized to peak is slightly over-provisioned — DRAM stays the limiter.